

赛题描述

赛题名称

面向 RTL 的 Agent 自动验证环境生成

赛题背景

RTL 设计验证通常需要工程师根据模块接口手工编写 testbench、driver、monitor、scoreboard、随机约束和覆盖率模型。对于 AXI、SRAM、valid/ready 流式接口等常见协议，这些工作具有一定规律，但仍然依赖大量人工经验。

本赛题希望参赛系统能够读取给定 RTL，自动分析模块接口和协议特征，生成可运行的验证环境，并通过约束随机测试和覆盖率反馈提升验证质量。

任务目标

给定一个 RTL 设计，参赛系统需要自动生成该设计的验证环境与分析结果。生成结果应至少覆盖以下能力：

- 解析 RTL 模块接口，识别时钟、复位、参数、端口方向、端口位宽和协议分组。
- 自动生成测试平台骨架，包括时钟/复位、输入驱动、输出监视和自动比较框架。
- 生成约束随机测试输入，支持固定 seed 和固定序列数量，保证结果可复现。
- 生成或更新功能覆盖率 bin，并根据覆盖率反馈产生更有效的测试序列。
- 运行仿真并输出 line coverage、branch coverage、functional coverage 和统一报告。

最终评测以 10 个电路为单位进行，每个电路独立运行并独立产出结果。具体评分口径见单独的打分细则

文档。

输入约定

每个评测样例的输入是一个 RTL benchmark。最小输入包含：

```
benchmark/  
  rtl/  
    dut.v
```

其中：

- `rtl/` 中包含待验证 RTL 源码，可能是单文件，也可能包含多个依赖文件。
- 顶层模块名可能由评测入口参数给出，也可能需要从 RTL 中识别。

评测 RTL 主要覆盖以下接口类型：

- AXI4-Lite 或 AXI/AXI-like 接口。
- 简单 SRAM 类接口。
- 流式 valid/ready 接口。

输出约定

参赛系统应为每个输入电路生成一个独立结果目录：

```
submission_out/  
  case_name/  
    design.json  
    verification_skeleton.json  
    constraints.json  
    coverage_bins.json  
    functional_coverage.json  
    coverage_result.json  
    report.json  
    generated_tb/
```

generated_tests/

design.json

描述 RTL 设计的接口解析结果，应包含：

- 顶层模块名。
- RTL 文件列表。
- 时钟和复位信号。
- 端口方向、位宽、类型和协议分组。
- 推断出的接口协议，例如 AXI、SRAM、valid/ready stream。

verification_skeleton.json

描述自动生成的验证骨架，包含：

- 时钟/复位生成方式。
- driver 列表及其负责的接口。
- monitor 列表及其采样时机。
- DUT 输出结果。
- 生成的 testbench 源码路径。

该文件是证明生成的文件不是空壳。

constraints.json

描述约束随机测试策略，应包含：

- 固定 seed。
- 固定测试序列总数量，5000。
- 随机变量列表。

- 覆盖率反馈对约束的调整记录。

coverage_bins.json

描述功能覆盖率目标，应包含：

- coverpoint 名称。
- bin 定义。
- 每个 bin 对应的协议场景或边界场景。
- 未覆盖 bin 的反馈记录。

功能覆盖率 bin 既用于指导测试生成，也作为最终评分指标。每个 bin 必须有明确的采样条件，并在结果中记录命中次数。

functional_coverage.json

描述功能覆盖率采样结果，应包含：

- 已覆盖 bin 数、有效 bin 总数和功能覆盖率百分比。
- 每个 coverpoint 下各 bin 的命中次数和 covered 状态。

该文件中的 bin 名称和层次必须与 **coverage_bins.json** 对应。命中次数大于 0 的 bin 才视为已覆盖。

coverage_result.json

描述覆盖率结果，应包含：

- line coverage 值。
- branch coverage 值。
- functional coverage 值。
- 按打分细则计算的综合覆盖率。

report.json

顶层汇总报告，建议包含：

- 各阶段是否成功：解析、骨架生成、仿真和覆盖率收集。
- 关键输出文件路径。
- 覆盖率汇总。
- 可复现命令。

运行入口

参赛提交应提供一个统一运行入口，推荐形式类似于：

```
./run.sh \  
  --rtl benchmark/rtl \  
  --top dut \  
  --out submission_out/case_name \  
  --seed x \  
  --num-seq 5000
```

如果使用 Python 入口，也应提供等价命令：

```
python3 run.py \  
  --rtl benchmark/rtl \  
  --top dut \  
  --out submission_out/case_name \  
  --seed x \  
  --num-seq 5000
```

入口程序需要完成从 RTL 解析到结果输出的完整流程。不得要求评测人员手动修改生成文件。

环境约束

评测环境以官方提供的 Docker 或服务器环境为准。参赛系统应满足以下约束：

- 操作系统：Linux x86_64。
- Python：建议兼容 Python 3.10 或以上。
- 允许使用 Z3、PyVerilog 等基础开源库，但需要在提交中声明依赖。

- 运行过程必须可复现，同一 seed 和同一 RTL 输入应产生一致或等价的测试结果。
- 对于多文件 RTL，必须保留正确的编译顺序和 include 路径。

如果工具需要额外依赖，应提供 `requirements.txt`。

公开数据集

公开数据集位于 `public_dataset/`。其中 `case1` 到 `case5` 是公开样例，用于展示推荐的结果目录结构和报告格式。

公开数据集的作用：

- 帮助参赛者理解输入 RTL 和输出报告的组织方式。
- 展示验证骨架、约束、功能覆盖率 bin 和三类覆盖率结果的 JSON 表达。
- 提供可参考的 cocotb/VCS 覆盖率采集流程。
- 作为本地调试和格式对齐样例。

公开数据集不代表隐藏评测的完整范围。隐藏评测会使用同一类输入输出约定，但 RTL 结构、参数、接口组合和 corner case 可能不同。

评测流程

每个隐藏电路的评测流程如下：

1. 评测系统提供 RTL 输入目录、顶层模块名、seed 和序列数量。
2. 调用参赛系统的统一运行入口。
3. 检查输出目录结构和 JSON 文件格式。
4. 编译并运行生成的 testbench。
5. 收集 line/branch coverage，并检查 functional coverage 的 bin 命中结果。

- 6. 生成单电路评测结果。
- 7. 对 10 个电路的结果求和，得到最终成绩。

验证骨架是基础门槛。如果某个电路的 testbench 无法编译、无法启动仿真、无法正确连接 DUT，或者没有有效 driver/monitor/scoreboard，则该电路后续覆盖率不再计入。

打分细则

总分 100 分，共 10 个隐藏电路，每个电路 10 分。

总体结构

总分 = sum(单电路得分)，共 10 个电路
单电路得分 = 验证骨架得分 + 覆盖率得分

项目	分值
验证骨架生成	3 分
行覆盖率 + 分支覆盖率 + 功能覆盖率	7 分

验证骨架是门禁项。若某个电路的验证骨架无法编译、无法运行、无法正确连接 DUT，或不能完成基本驱动、监视和结果比较，则该电路直接记 0 分，不再计算覆盖率得分。

验证骨架，3 分

验证骨架应能解析 RTL 接口，并生成可运行的 testbench。至少支持 AXI/AXI-Lite、SRAM-like、valid/ready stream 等评测所用接口。

子项	分值	要求
RTL 接口解析	0.8	正确识别端口方向、位宽、时钟、复位、参数和接口分组
时钟与复位	0.6	时钟稳定，复位时序正确，复位后信号进入可用状态
输入驱动与输出监视	0.8	driver/monitor 的握手、采样时机和 backpressure 处理正确
自动比较框架	0.8	reference model 或 scoreboard 能比较 DUT 结果并报告失败原因

以下任一情况视为门禁失败：

- testbench 无法编译或仿真无法启动。
- DUT 端口漏连、错连或位宽不匹配。
- 时钟、复位或关键协议握手不可用。
- 没有有效 driver/monitor，无法产生和采样事务。
- scoreboard 不比较 DUT 行为，或静默忽略比较失败。

覆盖率，7 分

评测使用一次完整、可复现且仿真通过的结果，统计行覆盖率、分支覆盖率和功能覆盖率。

综合覆盖率按以下公式计算：

$$C = 0.4 * \text{行覆盖率} + 0.3 * \text{分支覆盖率} + 0.3 * \text{功能覆盖率}$$

行覆盖率与分支覆盖率原有的相对权重仍为 60:40；两者合计占综合覆盖率的 70%，功能覆盖率占 30%。三项均使用百分数，例如 96.90 表示 96.90%。

功能覆盖率按有效 bin 的命中比例计算：

$$\text{功能覆盖率} = \text{已命中且命中次数大于 0 的有效 bin 数} / \text{有效 bin 总数} * 100\%$$

同一个 bin 命中多次仍只计为一个已覆盖 bin。功能 bin 必须对应可解释的功能场景、边界条件、协议状态或场景组合；仅把随机值逐个枚举成 bin、无法由测试采样证明命中的 bin，不计入有效总数。

综合覆盖率 C	覆盖率得分
$C \geq 85\%$	7.0 分
$65\% \leq C < 85\%$	4.9 分
$45\% \leq C < 65\%$	2.8 分
$C < 45\%$	0 分

有效运行要求：

- 提供固定 seed 和固定测试规模，保证结果可复现。
- `constraints.json` 描述随机变量、范围、协议约束和 seed。
- `coverage_bins.json` 定义 coverpoint、bin 及对应功能场景。
- `functional_coverage.json` 记录每个 bin 的命中次数、已覆盖状态和汇总比例。
- `coverage_result.json` 同时包含 line、branch、functional 及综合覆盖率。
- VCS/URG 报告必须对应当前提交的 RTL；cocotb 与纯 Verilog testbench 均可使用。

缺失或无效结果按以下规则处理：

- 仿真失败、挂死或 DUT 无法编译：覆盖率得 0 分。
- 缺少某一项覆盖率：缺失项按 0% 计算。
- 功能 bin 无采样依据、结果不可解析或与定义不一致：功能覆盖率按 0% 计算。
- 没有可复现 seed 或输入序列：覆盖率项最高不超过 3 分。
- 报告与当前 RTL 不对应：覆盖率得 0 分。

示例：public_dataset/case4

case4 使用 cocotb 驱动 5000 条固定 seed 的 AXI 事务，VCS/URG 收集行覆盖率和分支覆盖率，cocotb 同时采样功能 bin。结果为：

- 行覆盖率：125 / 129 = 96.90%
- 分支覆盖率：25 / 31 = 80.65%
- 功能覆盖率：29 / 29 = 100.00%

综合覆盖率为：

$$\begin{aligned} C &= 0.4 * 96.90 + 0.3 * 80.65 + 0.3 * 100.00 \\ &= 92.96\% \end{aligned}$$

$C \geq 85\%$ ，因此覆盖率得 7 分。若验证骨架门禁通过并得到 3 分，则：

$$\text{case4 得分} = 3 + 7 = 10 / 10$$

最终成绩为 10 个电路得分之和。